Planificación de procesos: Introducción

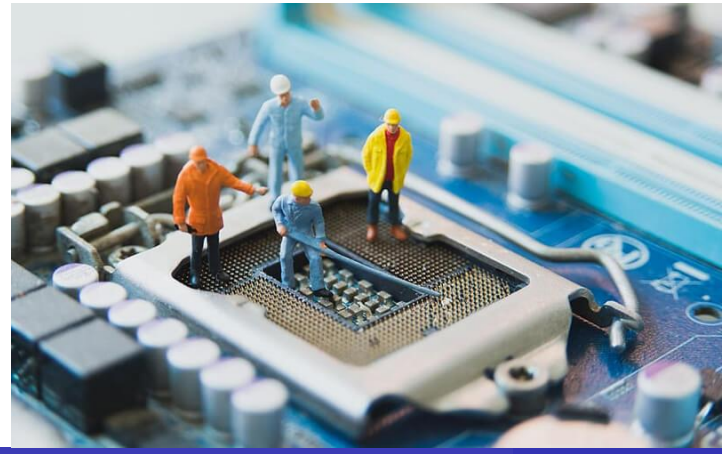
Sistemas Operativos I

Oliver Díaz

Grau d'Enginyeria Informàtica

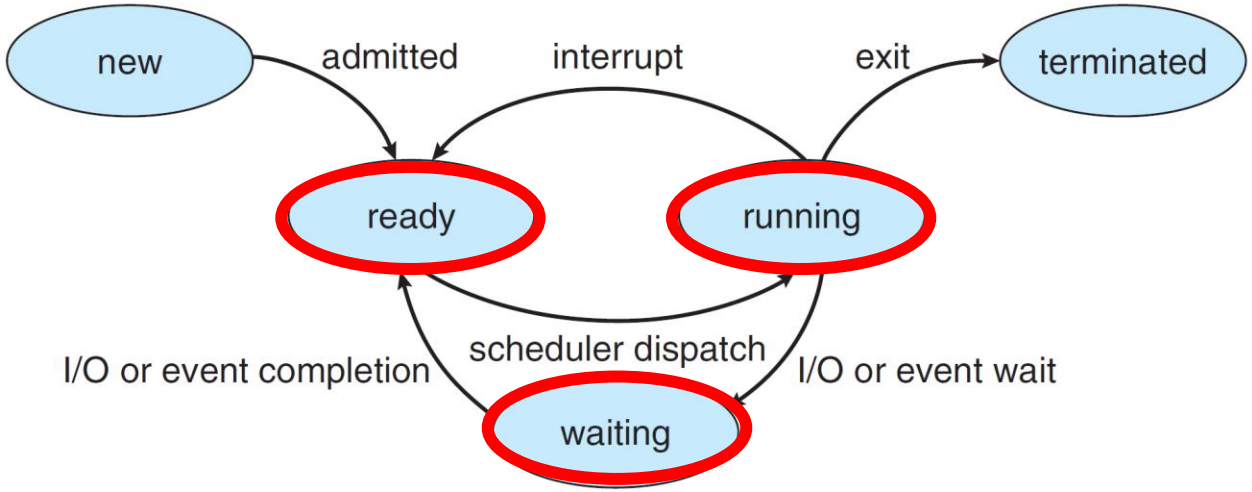
Definición

- La **planificación de procesos (scheduling)** es una de las funciones más importantes del SO, ya que manejamos **recursos limitados**.
- Herramienta mediante el cual el SO decide el orden en el que se ejecutarán los procesos activos.
- Asigna tiempo de CPU de manera “equitativa” a los procesos activos del sistema. Se pretende maximizar el rendimiento y garantizar que todos los procesos se ejecuten de manera “justa”.



Estado de un proceso

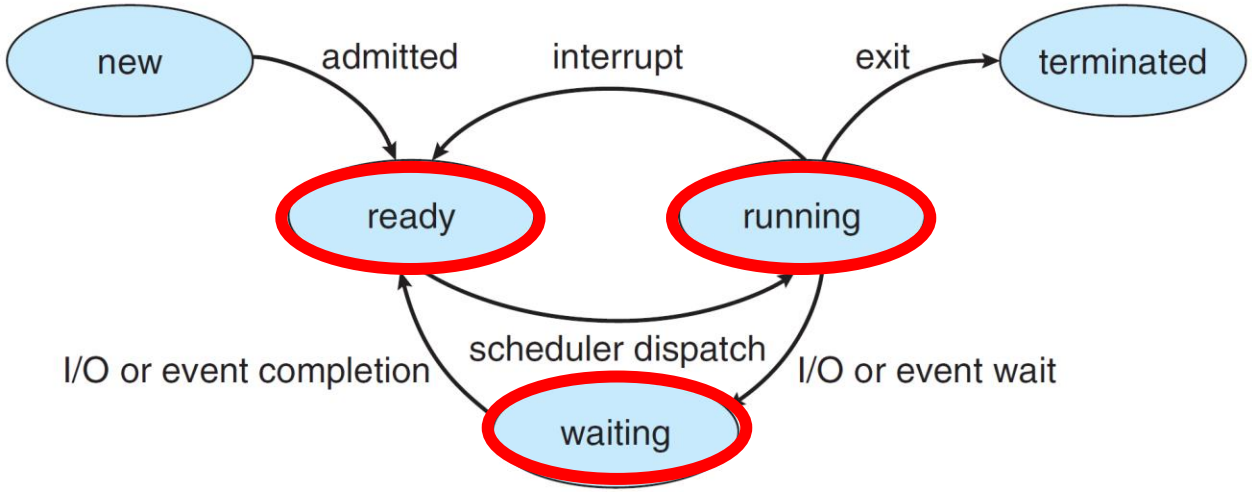
- A medida que el proceso se ejecuta, su estado cambia. El **estado** de un proceso viene determinado por su actividad actual
  - Nuevo/*New*: ha sido creado, pero no admitido por el SO
  - Listo/*Ready*: en espera de que se le asigne tiempo CPU para ejecutarse (**candidato del SO/planificador a ejecutarse**)
  - Ejecución/*Running*: esta siendo ejecutado por la CPU
  - En espera o bloqueado/*Waiting*: en espera de algun evento externo
  - Terminado o Salida/*Terminated* o *Exit*: finalización de ejecución



Silberschatz, A., Peterson, J. L., & Galvin, P. B. (2018). *Operating system concepts*. Wiley

Estado de un proceso: Listo <-> Ejecución

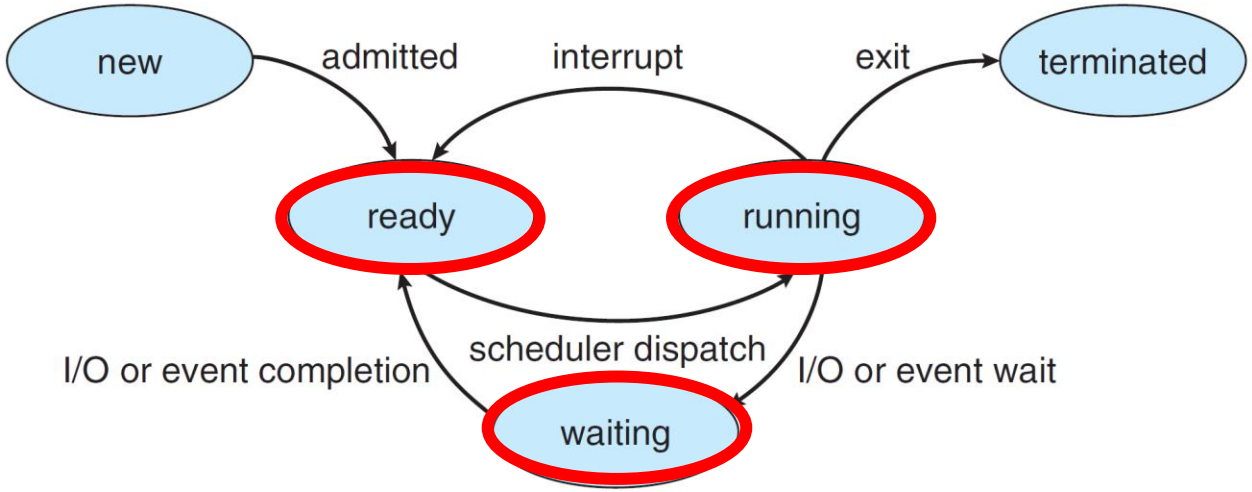
- En general, el SO (a través del planificador) decide cuando un proceso deja de ejecutarse y pasa al estado *Listo/Ready* (y a la inversa).
- Un proceso también puede voluntariamente dejar de ejecutarse y pasar a un estado *Listo/Ready* (función `sched_yield` de C).
  - Realiza tarea que no requiere toda la potencia de la CPU
  - Espera a que ocurra un evento externo



Silberschatz, A., Peterson, J. L., & Galvin, P. B. (2018). *Operating system concepts*. Wiley

Estado de un proceso: bloqueado

- Un proceso esta *En espera/Bloqueado/Waiting* si esta esperando 1 evento.
  - Llamada a función de lectura para leer datos de red, teclado o disco. Hasta que los datos no estén disponibles, el proceso esta bloqueado
  - Se duerme voluntariamente mediante un `sleep` o `wait` (espera por 1 proceso)
- Este proceso *Bloqueado* no es elegible de ser ejecutado por el SO hasta que no tenga lugar este evento. Entonces pasará a un estado *Listo/Ready*.



Silberschatz, A., Peterson, J. L., & Galvin, P. B. (2018). *Operating system concepts*. Wiley

Conceptos

- **Cambio de contexto**: mecanismo mediante el cual el SO (núcleo) cambia la ejecución de un proceso por otro.

- **Planificador**: algoritmo que utiliza el SO para gestionar los procesos y decidir cual es el siguiente proceso a ejecutar. Cara SO puede utilizar una técnica (e información) diferente para tomar esta decisión.

Conceptos

- El **Bloque de Control de Procesos** (*Block Control Process* BCP) es una estructura de datos utilizada por el SO para representar y controlar cada uno de los procesos que se están ejecutando en el sistema.
- Contiene información detallada sobre cada proceso necesario para iniciar o reiniciar un proceso, etc.
- Utilizado por el SO para:
  - Planificar el uso de la CPU
  - Cambiar de contexto
  - Gestión de procesos en espera
  - Asignación de recursos
  - Terminación de procesos

Estado
Identificador (PID)
Prioridad
Tiempo CPU utilizado
Registro CPU
…

Gracias

Planificación de procesos: Cambio de contexto

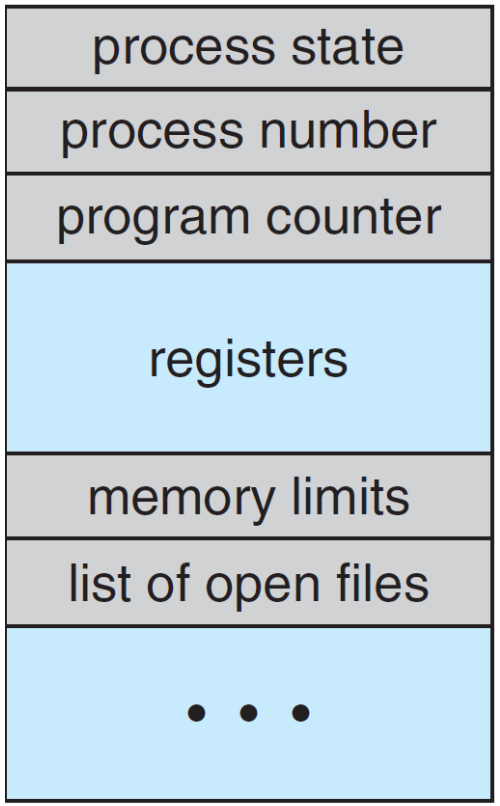
Sistemas Operativos I

Oliver Díaz

Grau d'Enginyeria Informàtica

Cambio de contexto

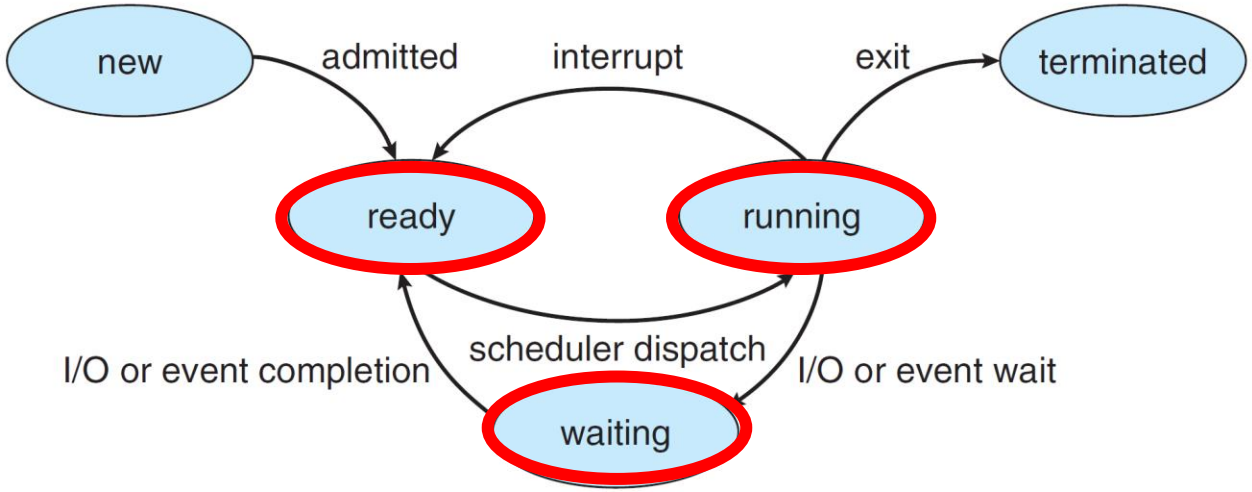
- Cada proceso esta representado en el SO por BCP.
- Para que se pueda realizar el cambio de contexto, el **BCP** almacena para cada proceso:
  - PID (un entero en UNIX)
  - Estado del proceso (*Listo*, *Ejecutando*, *Bloqueado*…)
  - Registros de CPU cuando este *Listo* o *Bloqueado*
  - Privilegios y/o prioridades
  - Memoria física donde reside, fichero ejecutable,…
  - Tiempo COU, tiempo real, limites tiempo,…



Silberschatz, A., Peterson, J. L., & Galvin, P. B. (2018). *Operating system concepts*. Wiley

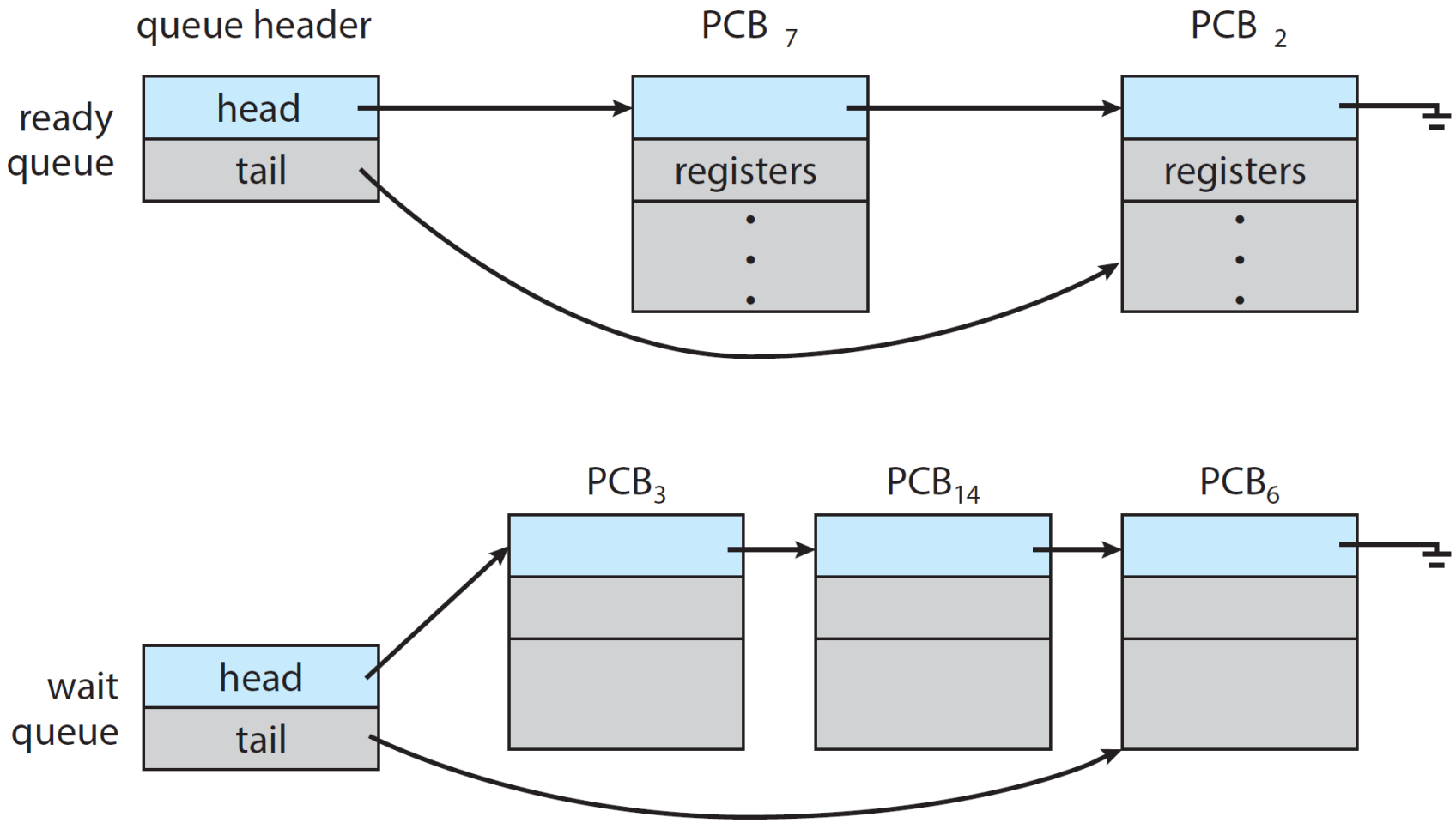
Cambio de contexto

- Al pasar un proceso de modo *Ejecución* a *Listo* o *Bloqueado*, se produce un **cambio de contexto**.
  - Para dejar de ejecutar un proceso, el SO guarda toda la información del proceso que se está ejecutando en su BCP.
  - Una vez el SO elige un proceso a ejecutar, se restauran los valores de registros almacenados en su BCP. Estos valores se copian a la CPU y el proceso continua ejecutándose dónde lo había dejado anteriormente.



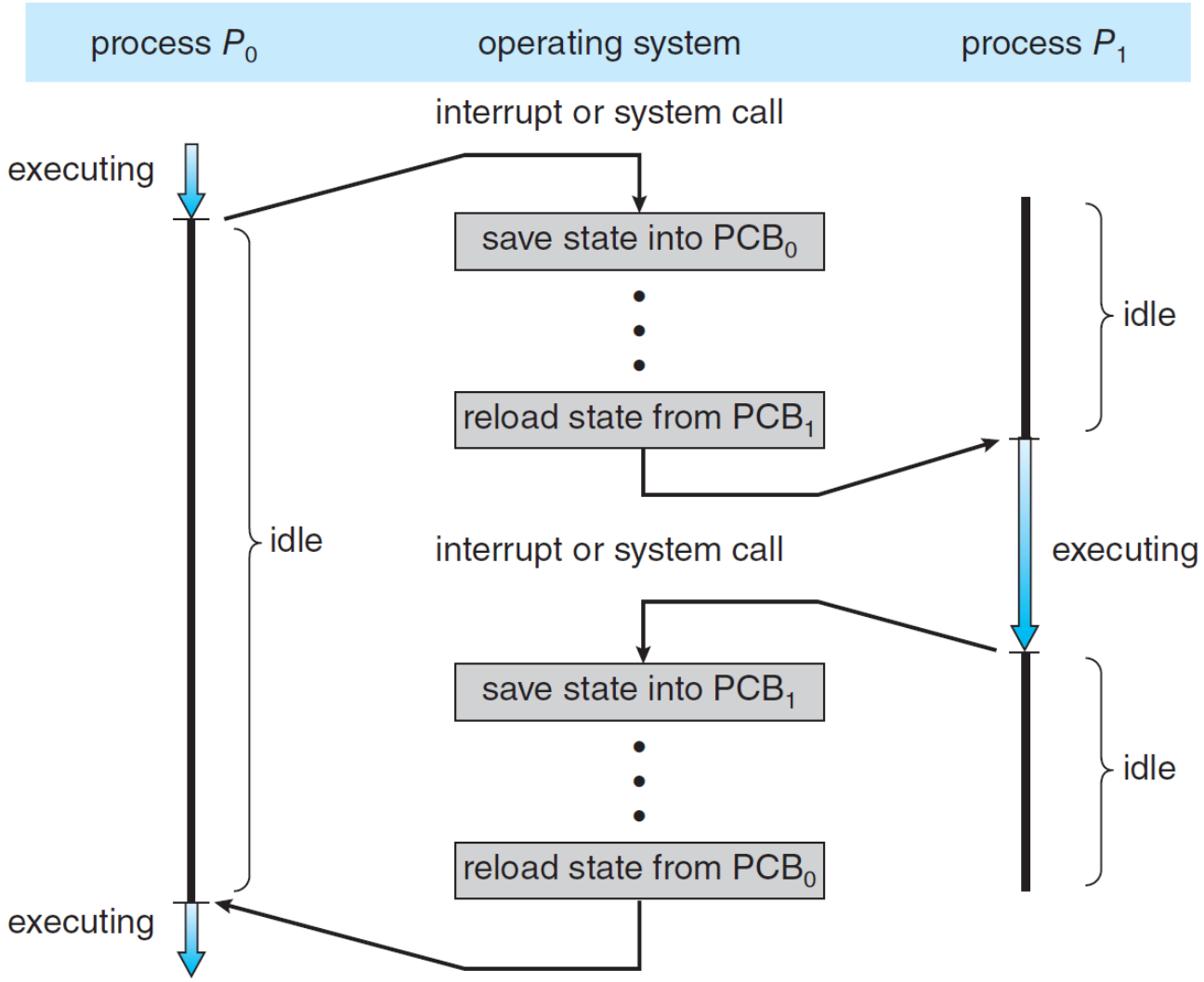
Silberschatz, A., Peterson, J. L., & Galvin, P. B. (2018). *Operating system concepts*. Wiley

Colas procesos “Listos” y “Bloqueados”



Silberschatz, A., Peterson, J. L., & Galvin, P. B. (2018). *Operating system concepts*. Wiley

Cambio de contexto

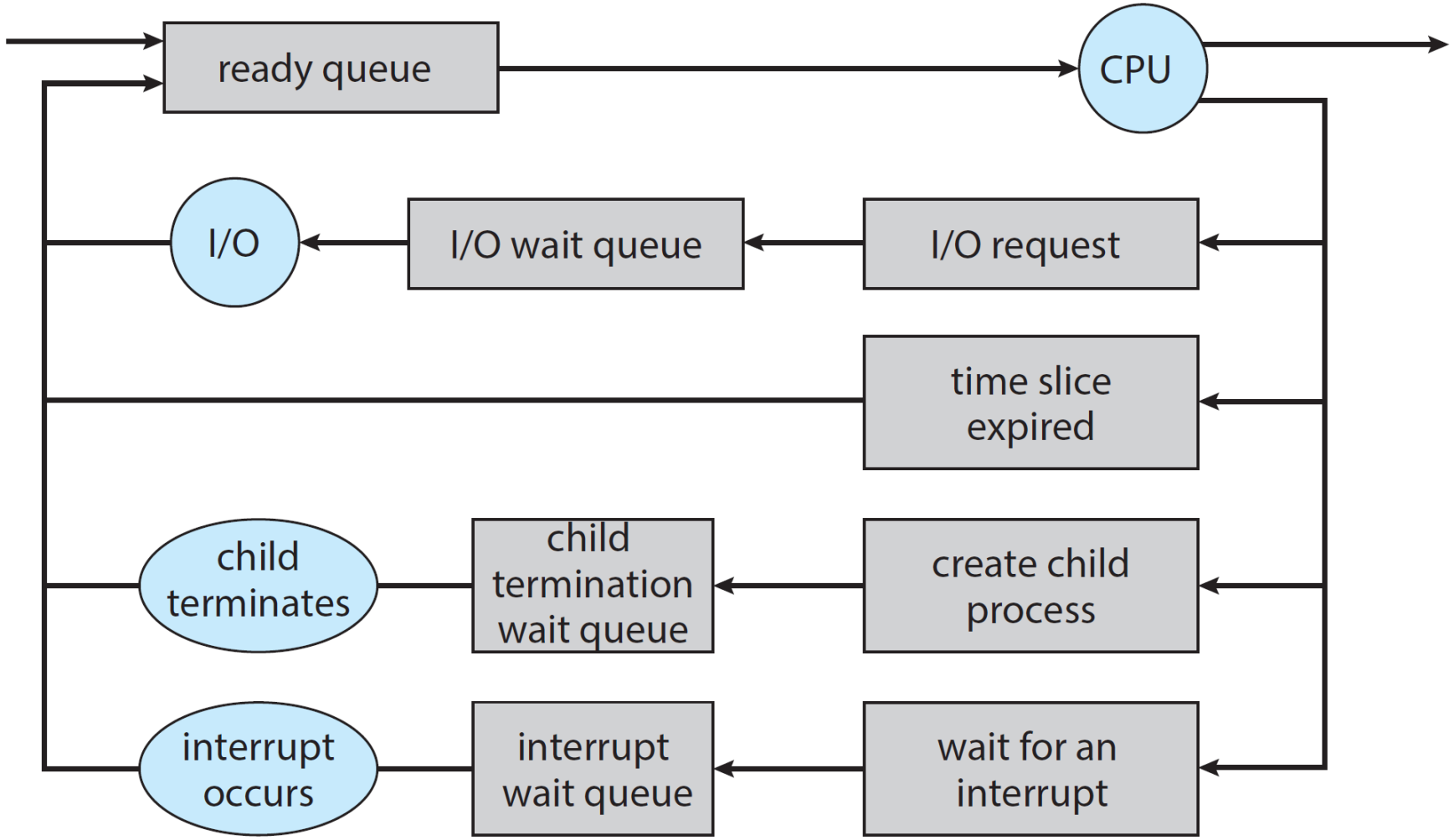


Silberschatz, A., Peterson, J. L., & Galvin, P. B. (2018). *Operating system concepts*. Wiley

¿En que situaciones se produce un cambio de contexto?

- Al producirse una **interrupción de temporizador** (p.ej. ~100 ms). El planificador decide dejar de ejecutar el proceso actual y pasa a ejecutar otro.
- Cuando se **bloquea un proceso**. En ese momento, el planificador decide cual de los procesos con estado *Listo* pasa a ejecutarse.
- Con **otros tipos de interrupción.** Cuando llegan datos por la red, al haberse leído datos en disco, etc.. se interrumpe la ejecución de un proceso. El planificador puede decidir ejecutar el proceso que se solicitado datos o continuar con la ejecución del proceso que se estaba ejecutando en aquel momento.
- Otros casos.

¿En que situaciones se produce un cambio de contexto?



Silberschatz, A., Peterson, J. L., & Galvin, P. B. (2018). *Operating system concepts*. Wiley

Ejemplo: interrupción de temporizador

- Cuando ocurre una interrupción de temporizador en un SO, se produce un cambio de contexto para permitir que el planificador de procesos seleccione el siguiente proceso que se ejecutará.

- Este proceso de cambio de contexto debido a una interrupción de temporizador se repite continuamente en el SO, permitiendo que los procesos se ejecuten de manera equitativa y que el sistema utilice los recursos de manera eficiente.

Pasos a seguir: interrupción de temporizador

1. El controlador de interrupciones del SO se activa debido y detiene la ejecución del proceso.
2. Se invoca al planificador de procesos para determinar qué proceso debe ejecutarse a continuación.
3. El planificador de procesos selecciona el siguiente proceso que se ejecutará a partir de la lista de procesos Listos.
4. El estado del proceso seleccionado (PCB) se carga en la CPU y se restaura su contexto.
5. El proceso seleccionado se ejecuta en la CPU hasta que ocurre otro cambio de contexto o hasta que finaliza su ejecución.

Gracias

Planificación de procesos: Algoritmos de planificación

Sistemas Operativos I

Oliver Díaz

Grau d'Enginyeria Informàtica

Planificación

- ¿Cómo elige el SO (planificador) el proceso a ejecutar a continuación? ¿Cuál es el mejor método?
- La solución más “justa” podría ser una cola FIFO circular.
  - En ciertas ocasiones, puede ser la peor opción ya que el usuario no percibe visualmente una buena respuesta
- Hoy día los ordenadores trabajan a velocidades muy altas y podríamos pensar que este tema no es importante.
  - Los servidores van muy cargados de tareas y hay que decidir bien el proceso
- No es una decisión trivial.

Tipos de carga

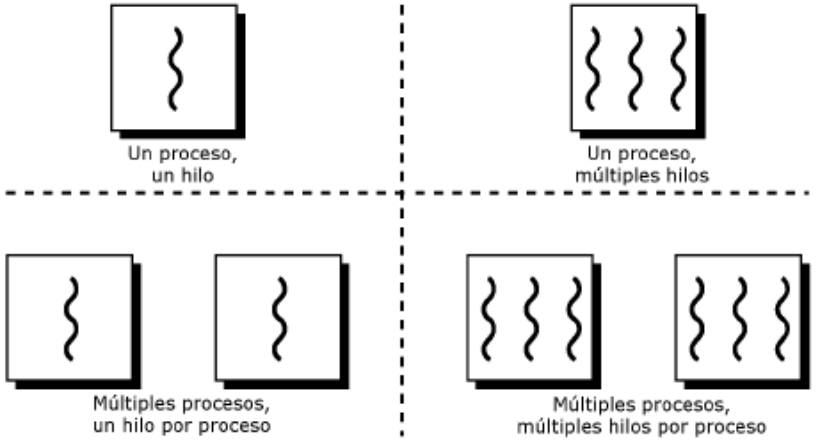
- Aquellos algoritmos de planificación que funcionan bien en entornos de alta carga computacional (CPU bound) pueden no ser idóneos en entornos con carga alta de entrada-salida (I/O bound), y al contrario.
  - ***CPU bound***. Pocos requerimientos de E/S, pero utiliza la mayoría del tiempo realizado cálculos computacionales.
  - ***I/O bound.*** Pasa más tiempo realizando tareas de E/S que computacional.

Algoritmos de planificación

- No existe un planificador optimo general.
- Cada uno tiene sus ventajas e inconvenientes.
- El objetivo de este tema es ver algoritmos que funcionan bien en entornos con carga variada, que utilizan los ordenadores personales.
  - Planificación uniprocesador
  - Planificación multiprocesador
  - Planificación de ejecución en tiempo real
  - Planificación en sistemas sobrecargados

Concepto: Tarea

- Un proceso puede ejecutar a lo largo de su ejecución múltiples tareas diferentes, cada una con características diferentes.
  - Realizar cálculos numéricos
  - Guardar datos en disco
  - Dibujar el menú por pantalla al hacer clic con el ratón
- Por eso, hablaremos de **tareas** y no de procesos
- Además, un proceso puede estar formado por muchos hilos de ejecución, cada uno ejecutando una tarea.



Gracias

Planificación de procesos: Planificación uniprocesador

Sistemas Operativos I

Oliver Díaz

Grau d'Enginyeria Informàtica

Algoritmos de planificación

- No existe un planificador optimo general.
- Cada uno tiene sus ventajas e inconvenientes.
- El objetivo de este tema es ver algoritmos que funcionan bien en entornos con carga variada, que utilizan los ordenadores personales.
  - **Planificación uniprocesador**
  - Planificación multiprocesador
  - Planificación de ejecución en tiempo real
  - Planificación en sistemas sobrecargados

Round robin (o FIFO circular)

- El planificador dispone de una lista de tareas y las ejecuta de forma ordenada (de manera circular).
- Cada una de las tareas se ejecutan durante un tiempo limitado (**rebanada de tiempo o *time slice***).
- Cada vez que finaliza una rebanada el planificador hace un cambio de contexto a otra tarea. La tarea se puede ejecutar en menos tiempo.
- Al realizar un cambio de contexto, se pone la tarea actual al final de la lista de tareas (en el estado *Listo*) y se coge la primera tarea de la lista de tareas *Listas* (y se pone en *Ejecución*)

Round robin (o FIFO circular)

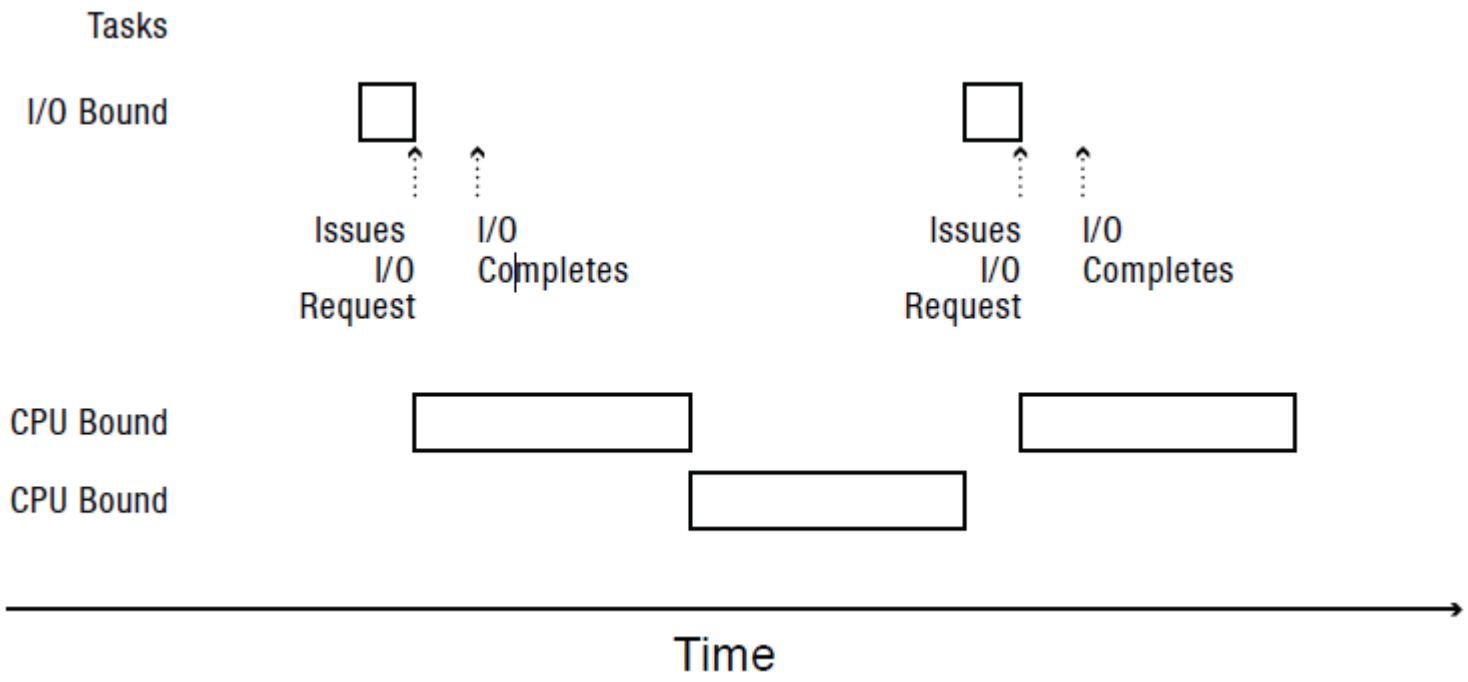
- Al realizar un cambio de contexto, se necesita guardar registros del proceso actual en ejecución, seleccionar nuevo proceso y cargar sus registros a la CPU (hoy día tarda unas decenas de microsegundos)
- Hay que tener en cuenta el impacto en la **memoria caché**.
  - La tarea seleccionada para ejecutar puede haber tenido (o no) datos en la caché durante su rebanada de tiempo anterior. Todo dependen de lo que haya hecho las otras tareas en su "ausencia". Quizás la tarea en ejecución debe acceder a la RAM para que se vuelvan a copiar los datos a la caché y eso es muy lento.
- Debe haber un **compromiso entre el tiempo de respuesta y la sobrecarga** a la hora de elegir valor de la rebanada de tiempo.
- Actualmente está entre 10ms y 100 ms.

Round robin (o FIFO circular)

- Es crítico seleccionar **el valor de la rebanada de tiempo**
  - Si es muy **pequeña**, habrá una sobrecarga de planificación. Se perderá mucho tiempo en hacer cambios de contexto y las tareas no se podrán realizar (alta **sobrecarga**)
  - Si es muy **grande**, las tareas esperarán mucho tiempo para obtener su turno y poder ejecutarse (**tiempo de respuesta bajo**)

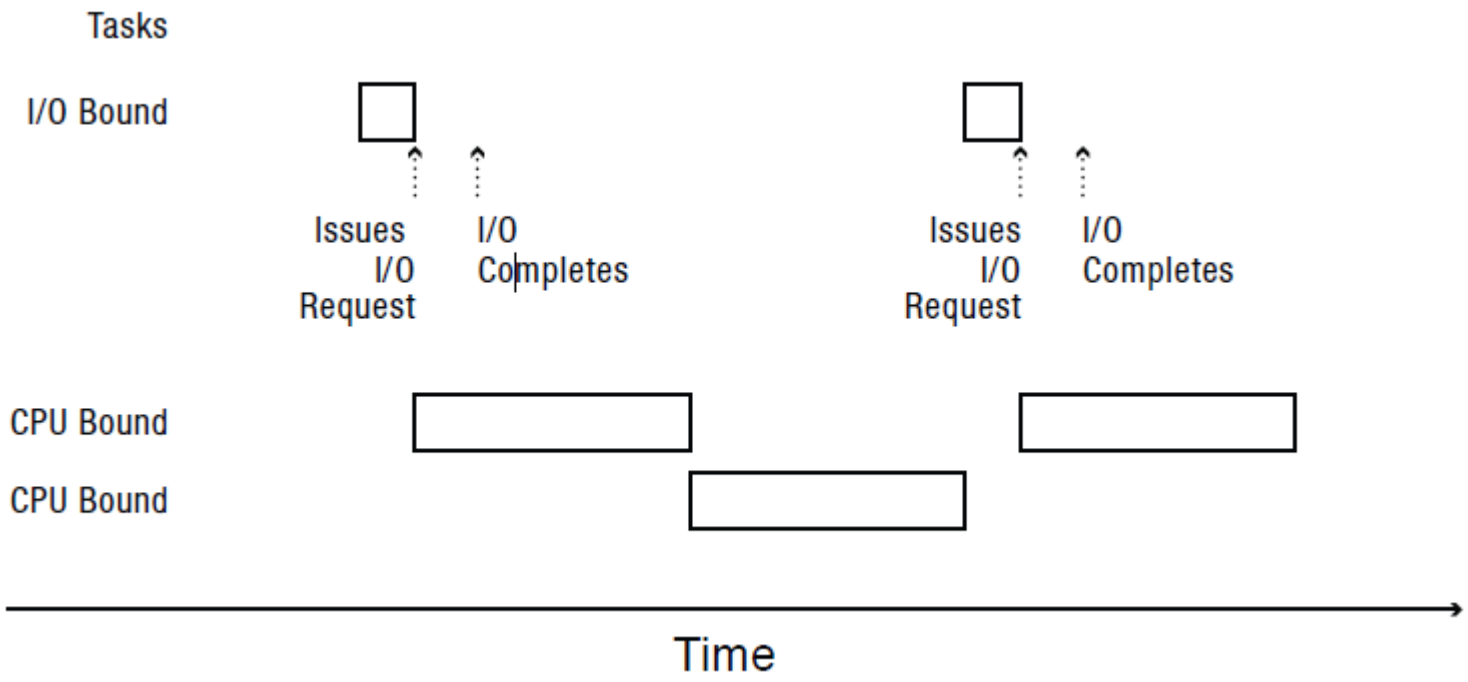
Round robin (o FIFO circular)

- La planificación *round robin* generalmente no es adecuada en un entorno con **carga variada**. Supongamos 3 tareas:
  - 2 *CPU bound*
  - 1 *I/O bound* que solo necesita la CPU entre 1 ms y 10 ms de disco



Round robin (o FIFO circular)

- Si la tarea *I/O bound* se ejecuta sola, podría tener ocupado el disco y acabar la tareas rápidamente.
- Con las otras dos tareas *CPU bound* y una rebanada de tiempo de 100ms, la tarea *I/O bound* se ralentizaría en un factor de 20. Solo necesita 1ms y luego debe esperar 200ms.
- Nos interesa priorizar *I/O bound* para que no espere tanto.



Round robin (o FIFO circular)

- *Round robin* no es buena solución si hay tareas *CPU bound* y *I/O bound*.
- Las tareas con gran volumen de E/S generalmente necesitan poco tiempo de CPU para realizar la siguiente operación E/S. Una vez emitida la operación, se realiza un cambio de contexto a otra tarea.
- En un editor de texto necesita pocos ms para mostrar por pantalla el valor de una tecla pulsada. Con una rebana de tiempo de 100ms, el editor de texto debería esperar múltiples rebanadas de tiempo para ejecutar y mostrar el resultado de sucesivas teclas.

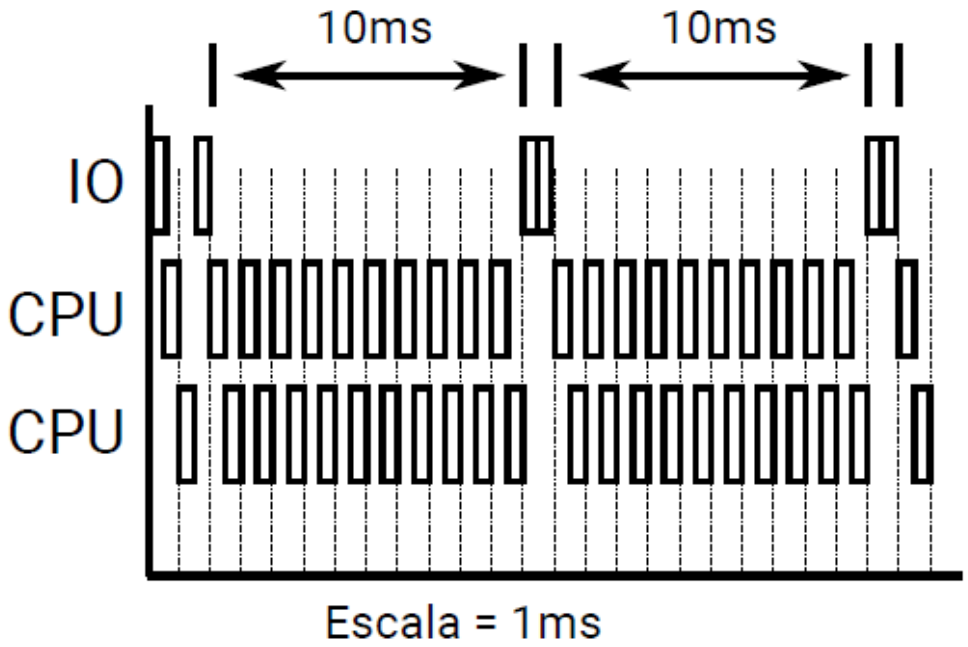
Round robin: ¿cuando usarlo?

- Adecuado si todas las tareas son del mismo tipo, en especial *CPU bound*.
- P.ej. Servidor de streaming de video (o música). Permite enviar todos los *streams* a una velocidad uniforme a pesar de que los datos se envíen a una velocidad más alta de la que necesita el receptor

- Entonces, ¿cómo se gestiona mejor las tareas CPU bound y I/bound?

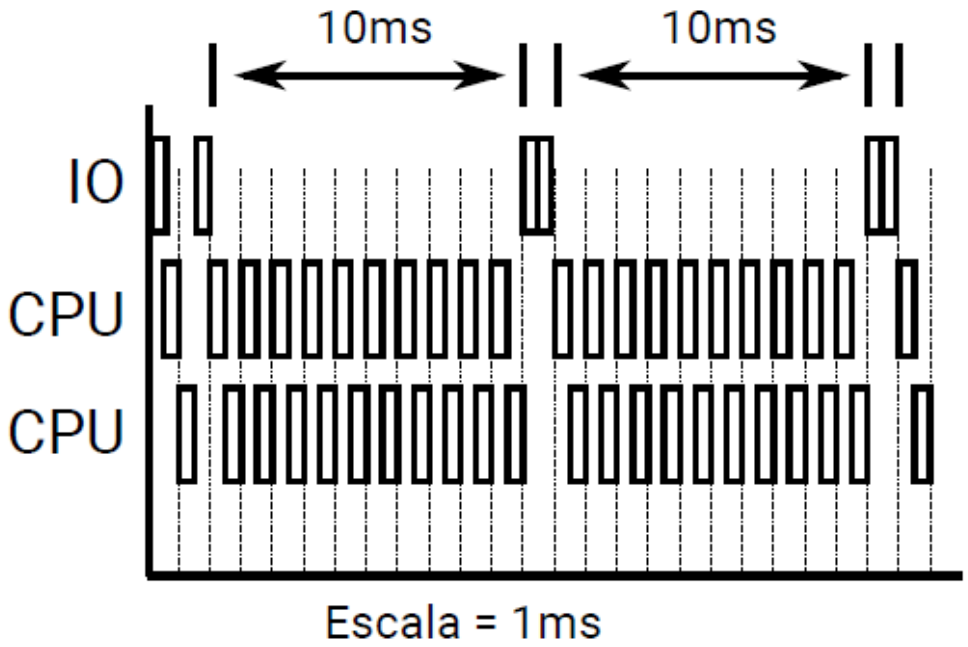
Max-min fairness

- El algoritmo **max-min fairness** se desarrollo inicialmente para la planificación de envío de paquetes por la red. Fue adoptado después para la planificación de tareas.
- La idea es asignar a cada instante de tiempo la CPU a una tarea *Preparada* que ha recibido menos tiempo de CPU.
- Supongamos el ejemplo anterior y que cada 0.5 ms hacemos un cambio de contexto en la tarea que ha recibido menos tiempo CPU.



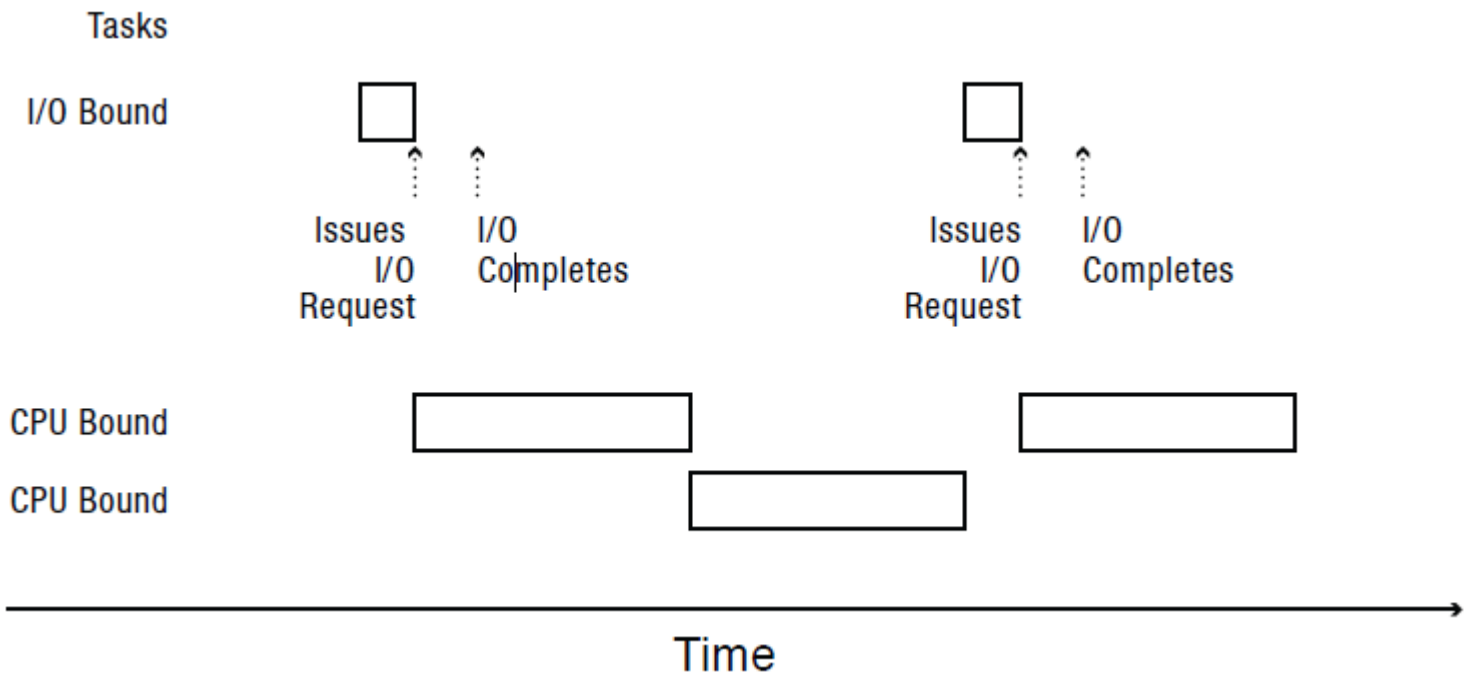
Max-min fairness

- A las tareas “pequeñas” se les asigna el tiempo que solicitan y reparten el resto del tiempo entre las tareas “grandes”.
- Los recursos son asignados en orden creciente de demanda y ninguna tarea tiene más recursos del que necesita.
- Si hay mezcla de tareas (*CPU y I/O bound*) resulta en una asignación más justa.
- Si solo hubieran tareas de computación seria sencillo, ya que todas las tareas obtienen el mismo tiempo de CPU (**round robin**)



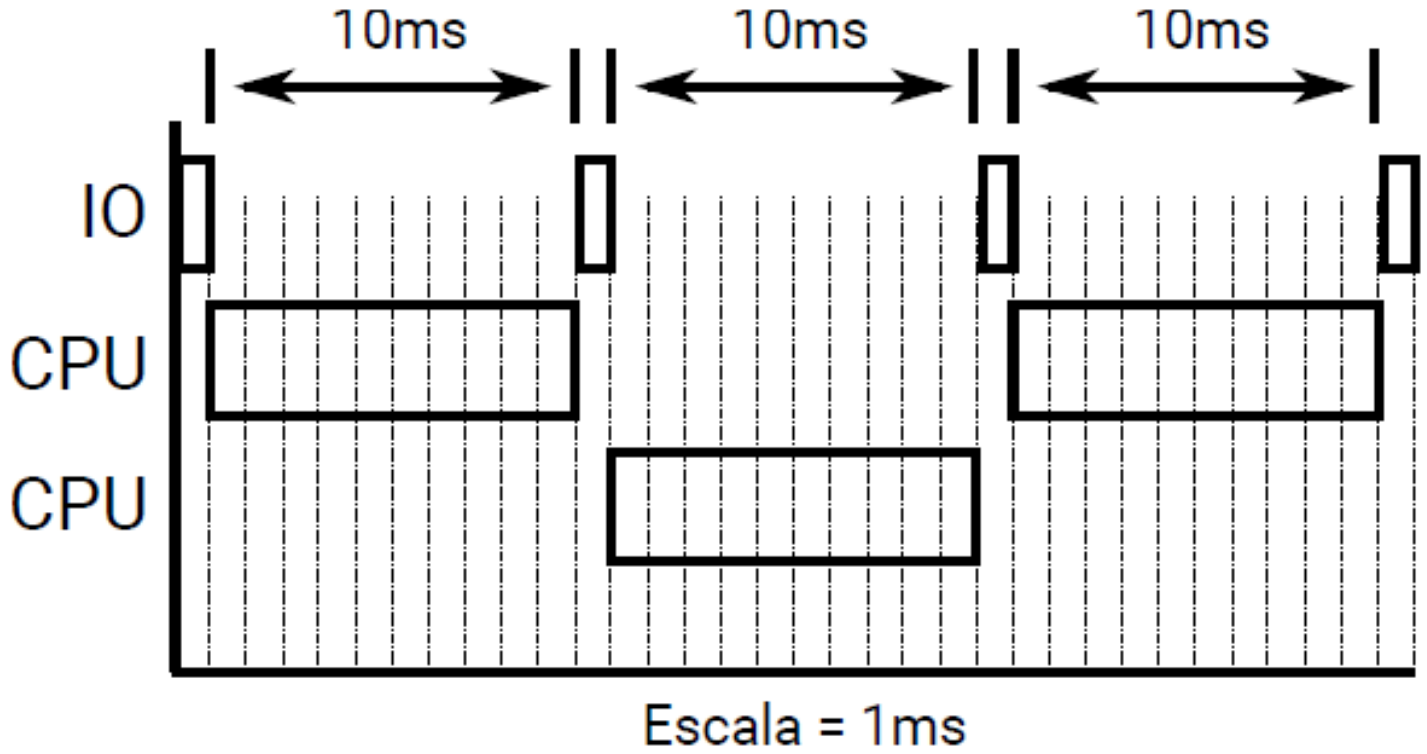
Round robin vs Max-min fairness

- Rebanada de tiempo de 100ms, 2 tareas *CPU bound*, 1 tarea *I/O bound* que solo necesita 1ms CPU y 10ms disco.
- En 201ms se ejecutan las 3 tareas: *I/O bound* recibe 0.5% CPU mientras que las otras dos reciben ~50% CPU.



Round robin vs **Max-min fairness**

- Al escribirse datos en disco se produce una interrupción y el SO puede decidir realizar un cambio de contexto.
- En este caso, aprox. 10% CPU se usa en la tarea *I/O bound* y 45% CPU entre las dos tareas *CPU bound*.
- Asignación más justa.



Max-min fairness

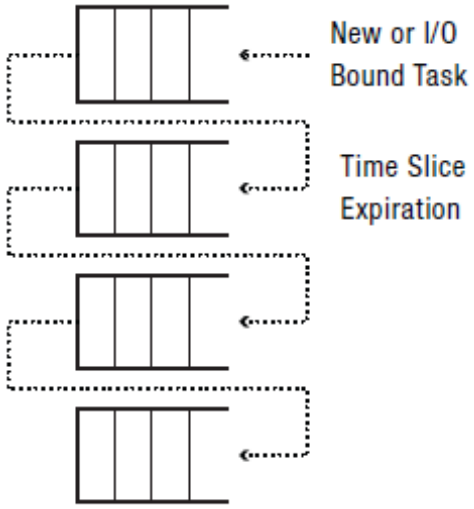
- En la práctica el max-min fairness se implementa (idealmente)…
- …cada vez que el planificador puede escoger una tarea a ejecutar, escoge aquella que tiene hasta ese momento el menos tiempo acumulado de CPU.
- Requiere de una **cola de prioridad** (¡alto coste computacional!)
- Un servidor puede requerir 100s o 1000s de decisiones para planificar cada segundo y no es conveniente mantener una cola de prioridad "ideal".
- Ahora veremos otro algoritmo simplificado…

Multi-level feedback queue (MFQ)

- Los SO **Win, Mac y Linux utilizan MFQ para planificar tareas**.
- Permite implementar una aproximación del algoritmo *max-min fairness* utilizando una extensión del *round robin*.
- Al MFQ hay múltiples colas *round robin* donde cada cola tienen una prioridad y una rebana de tiempo diferente.

Priority	Time Slice (ms)
1	10
2	20
3	40
4	80

Round Robin Queues

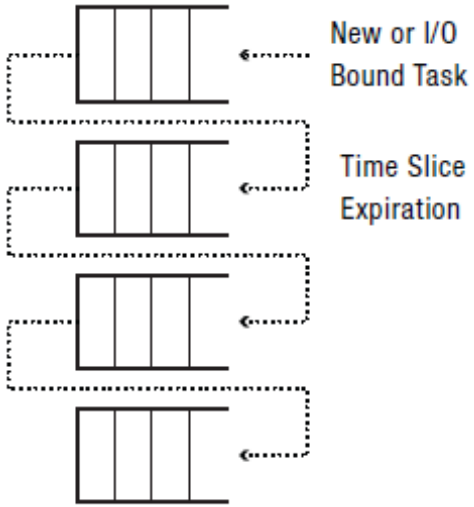


Multi-level feedback queue (MFQ)

- Las tareas de una cola del mismo nivel son planificadas de acuerdo a un *round robin*. Una tarea de prioridad alta se ejecuta antes que una de prioridad más baja.
- Las tareas se mueven por el planificador entre colas de prioridad para obtener equitatividad.
- Si una tarea consume su rebanada de tiempo (sin bloquearse), se mueve a una cola de prioridad más baja.
- Si se bloquea se incrementa la prioridad.

Priority	Time Slice (ms)
1	10
2	20
3	40
4	80

Round Robin Queues



Gracias

Planificación de procesos: Planificación multiprocesador, tiempo real y sobrec.

Sistemas Operativos I

Oliver Díaz

Grau d'Enginyeria Informàtica

Algoritmos de planificación

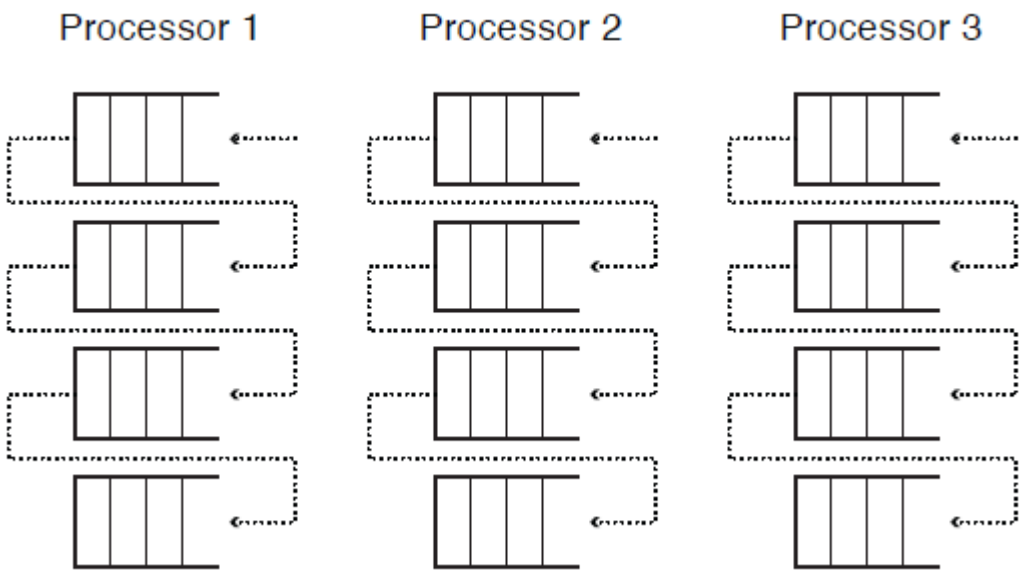
- No existe un planificador optimo general.
- Cada uno tiene sus ventajas e inconvenientes.
- El objetivo de este tema es ver algoritmos que funcionan bien en entornos con carga variada, que utilizan los ordenadores personales.
  - Planificación uniprocesador
  - **Planificación multiprocesador**
  - **Planificación de ejecución en tiempo real**
  - **Planificación en sistemas sobrecargados**

Planificación multiprocesador

- En la actualidad, los ordenadores, smartphones o tablets, … suelen disponer de sistemas con **múltiples procesadores**.
- ¿Como podemos utilizar de forma efectiva múltiples CPUs para ejecutar tareas secuenciales?
- ¿Cómo adaptamos los algoritmos de planificador uniprocesador a múltiples procesos?
- Debemos tener en cuenta que en los sistemas multiprocesadores, cada procesador tiene su propia memoria caché.

Planificación multiprocesador

- En SOs Win, Mac y Linux, cada CPU tiene su propia estructura MFQ, y por tanto, sus propias tareas.
  - Una tarea se ejecuta siempre en el mismo procesador para aprovechar los datos almacenados en la memoria caché.
  - Si un procesador se queda sin nada a ejecutar, puede “robarle” una tarea a otro procesador.



Planificación en tiempo real

- En determinados sistemas, el planificador tiene que asegurarse que determinadas tareas tienen **plazos** para ejecutarse.
- Por ejemplo:
  - **Avión**: El software asociado al control de vuelo debe ejecutarse antes en determinados momentos para que sea efectivo (¡y seguro para los pasajeros!)
  - **Coche**: El software asociado al bloqueo de frenos o control de tracción debe ejecutarse en el momento que sea accionado.
  - **Reproductor de vídeo**: Cada *frame* debe visualizarse cuando toca para evitar una visualización lenta del vídeo.

Planificación en tiempo real

- Existen diversos formas para intentar asegurarnos que se cumplen los plazos (¡aunque no hay seguridad al 100%!)
  - **Priorización por plazos**: Las tareas de tiempo real con plazos más cortos tienen una prioridad más alta y se les asignan recursos de forma preferente. Esta técnica funciona si las tareas son computacionales (*CPU bound*)
  - **Preempción**: Las tareas pueden ser interrumpidos en cualquier momento por tareas de mayor prioridad. Necesario para garantizar que las tareas críticas se ejecuten en el momento adecuado.

Planificación en sistemas sobrecargados

- ¿Qué pasa cuando el sistema se sobrecarga?
- Esto es especialmente importante en sistemas web, en los que el número de conexiones puede variar durante el día.
  - Muchos sistemas operan sin control directo sobre su carga y su rendimiento puede bajar mucho si hay sobrecarga.
  - Autopista con muchos coches.
  - Restaurante con muchos clientes y pocos camareros.

Planificación en sistemas sobrecargados

- Una solución sencilla seria **rechazar las peticiones de conexión.**
- Puede asegurar un buen servicio a las conexiones existentes (sistemas web de streaming de video)

Planificación en sistemas sobrecargados

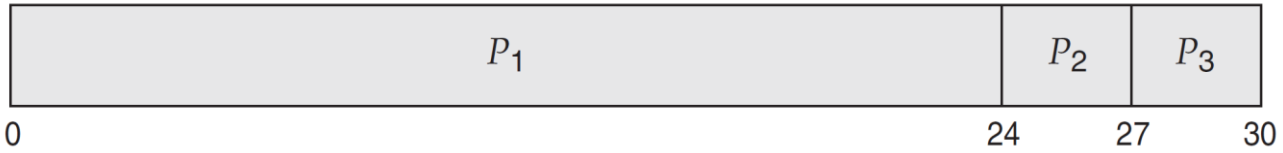
- Otra posibilidad para gestionar la sobrecarga es **reducir el tiempo de respuesta** para determinadas peticiones.
  - El 11S la página web de la CNN utilizó una página estática para informar.
  - Amazon utiliza un sistema que devuelve una página estática en caso que la información solicitada no este disponible a tiempo.
  - eBay actualiza con menos frecuencia la lista de subastas.
- Sin embargo, muchos sistemas hacen lo contrario: realizan más trabajo para tareas a medida que aumenta la carga.

Conclusiones express

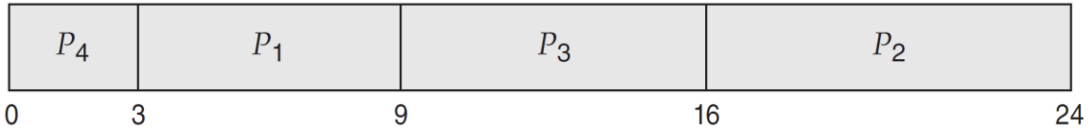
- La **planificación** es la técnica (algoritmo de software) que utiliza un SO para decidir cuál es el siguiente proceso en ejecutar.
- El **cambio de contexto** es el mecanismo hardware que se utiliza para pasar la ejecución de un proceso a otro.
- Un proceso puede tener (a grandes rasgos) los **estados** de *Listos*, *Bloqueados* o *En ejecución*.
- Existen múltiples **algoritmos de planificación**. ¿Cuál es el más adecuado? Depende del contexto en el que se ejecuten las aplicaciones.

Conclusiones express

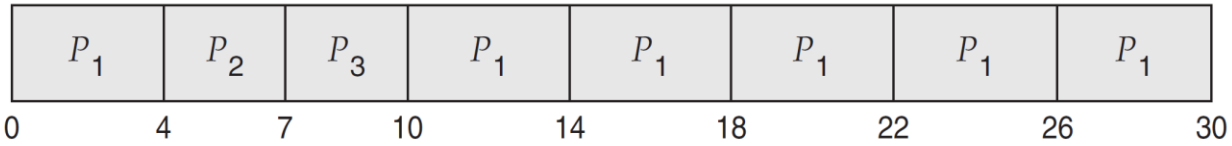
- Algoritmos de planificación
  - First-Come, First-Served



  - Shortest-Job-First

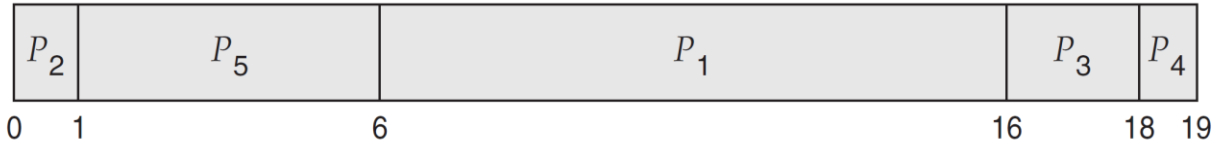


  - Round-Robin

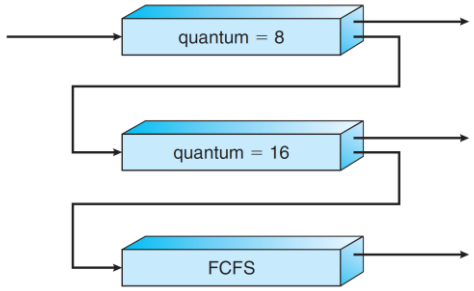


  - Prioridad

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2



  - Multilevel Feedback Queue
  - …



Gracias